

PROYECTO FINAL · CURSO DE IA

MealBoard

Planificación semanal de comidas, inventario y ahorro para personas que viven solas.

Lautaro Demonte · Desarrollo integral, producto, arquitectura, pruebas y documentación

Recurso	Acceso
Repositorio	github.com/LautyDe/Entrega-CursolA
Aplicación publicada	mealboard-ai.lauty-d-p.chatgpt.site
Demostración	Guion reproducible en docs/guion-demo.md; video opcional pendiente de grabación autenticada
Documentación	README.md, docs/arquitectura.md y anexos de evaluación

Versión documentada: 19 de agosto de 2026

1. Proyecto, problema y público

MealBoard resuelve una dificultad concreta: planificar una semana de comidas cuando una sola persona debe coordinar inventario, vencimientos, presupuesto, preferencias, habilidades de cocina y promociones. El costo de equivocarse no es solo económico: también puede producir desperdicio o una recomendación incompatible con una alergia.

La propuesta combina una interfaz web instalable con un ciclo de nueve agentes determinísticos. No utiliza un LLM ni una API paga en producción. Esto permite explicar cada decisión y conservar una prioridad rígida: alergias y alimentos vencidos prevalecen sobre gustos, ahorro o velocidad.

Resultado entregado

- Aplicación responsive/PWA con autenticación por email y Google, onboarding y datos por usuario.
- Calendario semanal configurable por categoría: balanceado, fit/proteico, vegano, vegetariano, sin TACC y delicioso.
- Inventario, lista de compras, confirmación de comidas cocinadas y descuento de ingredientes.
- Calorías y macronutrientes básicos estimados para lo planificado y lo cocinado.
- Calendarios comunitarios reales en D1, con descripción y público recomendado.
- Mapa de supermercados cercanos y consulta acotada de beneficios públicos oficiales.

Público objetivo

Personas adultas que viven solas en Argentina, cocinan con distintos niveles de experiencia y quieren reducir esfuerzo, gasto y desperdicio. La app evita exigir conocimientos técnicos y conserva confirmación humana para cambios importantes.

Alcance y honestidad de datos

Los precios del catálogo son demostrativos. Las publicaciones comunitarias sí provienen de usuarios autenticados. Las promociones públicas muestran fuente y vigencia cuando pueden verificarse, pero deben confirmarse antes de pagar porque cambian según entidad, sucursal y segmentación.

2. Arquitectura general

Separación entre interfaz, reglas agénticas, servicios tradicionales y persistencia.

Usuario PWA → React / Vinext UI y estado → API / Orquestador WorkingState → D1 + respaldo por usuario

Componente	Tipo	Responsabilidad
React + Vinext	Software tradicional	Interfaz, accesibilidad, validación visual y confirmaciones.
Orquestador	Control determinístico	Ejecuta nueve agentes en orden sin mutar el estado de entrada.
Agentes	IA simbólica / reglas	Observan, deciden y entregan resultados auditables.
Better Auth	Servicio de identidad	Email/contraseña, Google OAuth, sesión y recuperación.
Cloudflare D1	Persistencia principal	Usuarios, sesiones, estado privado y calendarios comunitarios.
localStorage	Respaldo local	Copia de continuidad; queda subordinada a D1 al autenticarse.
Overpass / OSM	Datos abiertos	Supermercados próximos, solo luego del permiso de ubicación.
Crawler oficial	Infraestructura acotada	Busca beneficios en dominios permitidos con límites y timeout.

La persistencia separa identidad y sesiones de los datos funcionales. Las rutas derivan la identidad desde la sesión y no confían en un identificador enviado por el navegador. La ubicación se usa de forma puntual y no se persiste.

3. Flujo de los nueve agentes

#	Agente	Observa	Decide / entrega
1	Captura	Perfil, inventario, comidas	Normaliza y separa urgentes/vencidos
2	Memoria	Evaluaciones previas	Recupera señales útiles
3	Análisis	Alergias, rechazos, equipos	Filtra recetas inseguras o inviables
4	Comunidad	Categoría/fuente real opcional	Transfiere orientación y etiquetas
5	Planificación	Candidatas, costo, urgencia	Asigna recetas a la semana
6	Recetas	Plan, nivel y equipos	Genera instrucciones adaptadas
7	Beneficios públicos	Medios y fuentes oficiales	Devuelve referencias verificables
8	Compras	Plan, stock, precios/promos	Calcula faltantes y costo
9	Evaluación	Plan completo	Revalida seguridad y presupuesto

Trazabilidad

Cada etapa conserva la forma **Observó / Decidió / Entregó**. El objeto WorkingState se copia en cada agente, de modo que la ejecución sea predecible y fácil de probar.

Barreras de seguridad

- Validación previa: elimina vencidos y recetas incompatibles con alergias, rechazos, nivel y electrodomésticos.
- Validación posterior: Evaluación vuelve a revisar el plan antes de mostrarlo.
- Confirmación humana: la propuesta no modifica el calendario persistido hasta que el usuario la acepta.
- Promociones: solo computan ahorro si coinciden proveedor, tipo de medio y vigencia.

4. UML de secuencia y memoria persistente

Paso	Usuario / UI	API y agentes	Persistencia
1	Configura perfil, stock y categoría	—	D1 guarda perfil confirmado
2	Solicita plan	API crea WorkingState	Recupera memoria previa
3	Espera propuesta	Agentes 1→9 + validaciones	Sin cambios definitivos
4	Revisa traza y calendario	Devuelve propuesta	—
5A	Confirma	API acepta resultado	D1 persiste plan/compras/memoria
5B	Cancela	Descarta propuesta	Estado anterior intacto
6	Marca comida cocinada	Valida ingredientes/cantidades	Descuenta inventario y suma nutrición

Entrada confirmada → Memoria previa → Propuesta segura → Confirmación → Aprendizaje

Qué se recuerda

- Perfil, preferencias, alergias, rechazos, equipos, presupuesto e inventario.
- Calendario confirmado, lista de compras, comidas cocinadas y totales nutricionales estimados.
- Evaluaciones y señales de semanas anteriores para ajustar variedad, velocidad y ahorro.
- El usuario puede consultar, corregir o eliminar la memoria desde la aplicación.

La memoria no autoriza a relajar una alergia. Las restricciones críticas se aplican nuevamente en cada ciclo.

5. Stack tecnológico y justificación

Tecnología	Uso	Por qué se eligió
TypeScript estricto	Dominio y contratos	Reduce estados inválidos entre agentes y API.
React 19	Interfaz	Componentes, estado y experiencia responsive.
Next.js 16 + Vinext	Rutas y compatibilidad	Modelo conocido de app/API con build sobre Vite.
Vite 8	Tooling	Desarrollo y compilación rápidos.
Cloudflare Workers	Runtime	Despliegue edge integrado con D1.
Cloudflare D1 + Drizzle	Datos	SQL administrado, tipado y adecuado al alcance multiusuario.
Better Auth	Autenticación	Email, OAuth, sesiones y hashes sin implementar criptografía propia.
Leaflet + OSM/Overpass	Mapa	Mapa accesible y datos abiertos sin servicio pago.
Resend	Emails	Verificación y recuperación; remitente sujeto a dominio autorizado.
Node test runner	Pruebas	Integración liviana con TypeScript y build real.
ESLint	Calidad	Reglas consistentes y detección temprana de errores.

Decisión central

El proyecto usa IA basada en reglas en lugar de un LLM. Para este dominio, la explicabilidad y el cumplimiento rígido de alergias y vencimientos tienen más valor que una respuesta creativa. También evita costo por token y dependencia de una clave paga.

6. Evidencia visual de producción

Captura real de la URL pública obtenida el 19 de agosto de 2026.

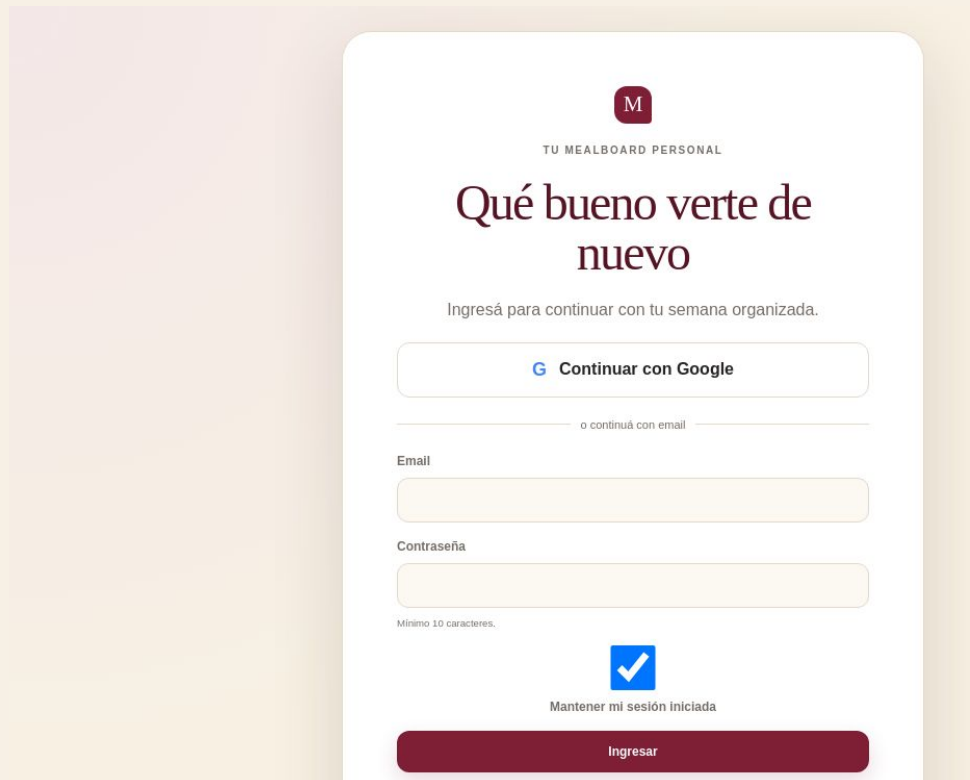


Figura 1. Pantalla pública de autenticación de MealBoard. Se observa identidad visual crema/bordo, alta por email y acceso con Google.

Qué demuestra

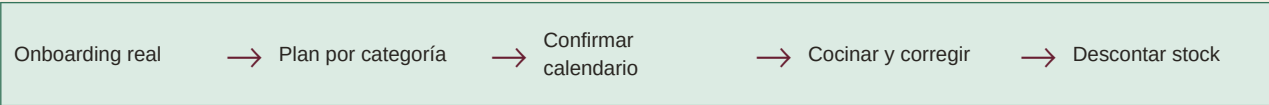
- La aplicación está desplegada y accesible públicamente.
- El acceso a datos personales está detrás de autenticación.
- La interfaz conserva jerarquía, etiquetas visibles y acciones clásicas.

Limitación de la evidencia

La captura automatizada disponible no tiene una sesión privada iniciada. Por privacidad, este informe no inventa capturas del interior. Las dos páginas siguientes documentan el flujo principal mediante artefactos técnicos reproducibles; el guion incluido indica cómo grabar la sesión autenticada opcional.

7. Evidencia del flujo principal

Representación técnica basada en rutas, estado y pruebas del repositorio.



Acción observable	Implementación	Resultado esperado
Completar perfil	POST /api/user-state	Estado asociado a la sesión en D1
Generar calendario	POST /api/plan	Propuesta + traza de nueve agentes
Confirmar	Estado UI + persistencia	Calendario aplicado solo con consentimiento
Marcar cocinada	lib/inventory-consumption.ts	Confirmación editable y descuento validado
Ver nutrición	lib/nutrition.ts	Calorías, proteínas, carbohidratos, grasas y fibra
Compartir	POST /api/community-calendars	Publicación real sin datos privados

Salida visible de IA

La salida principal es el calendario acompañado por la traza de decisiones. A diferencia de un chat generativo, cada agente declara qué observó, qué decisión tomó y qué entregó. Así se puede explicar por qué se descartó una receta o se priorizó un ingrediente próximo a vencer.

Confirmación requerida

La UI conserva la propuesta separada del calendario aplicado. Cancelar no modifica D1 ni el respaldo local; confirmar persiste plan, compras y memoria.

8. Evidencia reproducible y calidad

La sesión documentada recorre: autenticación → onboarding → inventario → categoría → agentes → confirmación → cocina → nutrición → comunidad → mapa/promociones. El detalle paso a paso está versionado en [docs/evidencias/sesion-real.md](https://docs.evidencias/sesion-real.md).

Cobertura automatizada

Suite	Qué verifica
rendered-html	Frontend renderizado, orquestación, restricciones y confirmación.
payments-and-persistence	Bancos, tipos de tarjeta, promociones compatibles y serialización.
promotion-discovery	Fuentes oficiales, extracción y límites de vigencia.
inventory-consumption	Corrección de cantidades, unidades y descuento sin stock negativo.
nutrition	Cálculo nutricional por receta, comida y seguimiento cocinado.

Comandos de aceptación

```
npm run lint
npm test
npm run sites:build
npm run sites:validate
```

Criterios de aceptación

- Cero errores de lint y build reproducible.
- Todas las pruebas pasan, incluidas seguridad alimentaria, persistencia, pagos y nutrición.
- El artefacto de hosting valida antes del despliegue.
- README, arquitectura, evidencias e informe corresponden al mismo commit.

9. Evaluación UX con Nielsen

Heurística	Estado	Evidencia
Visibilidad del estado	Cumple	Carga, ciclo de agentes, resultados y notificaciones.
Mundo real	Cumple	Alacena, vencimiento, compras y medios argentinos.
Control y libertad	Cumple	Confirmar/cancelar, corregir consumo y eliminar memoria.
Consistencia	Cumple	Patrones estables de campos, botones y estados.
Prevención de errores	Cumple	Barreras dobles para alergias/vencimientos y stock.
Reconocer antes que recordar	Cumple	Selectores, chips, categorías y etiquetas visibles.
Flexibilidad	Parcial	Responsive/PWA; faltan atajos e importación masiva.
Estética minimalista	Cumple	Crema, bordo, verde y tareas agrupadas.
Recuperación de errores	Cumple	Mensajes para incompatibilidad, stock y fallas externas.
Ayuda	Parcial	README y ayuda contextual; falta tour integrado.

Conclusión UX

Cumple ocho de diez heurísticas y cumple parcialmente dos. Para el público objetivo resulta especialmente importante que las decisiones de seguridad se expliquen y que ninguna acción irreversible dependa de recordar comandos.

10. Público objetivo y feedback real

La evaluación se realizó desde el caso de uso de una persona que vive sola, compra en supermercados argentinos y usa la app principalmente desde el celular. El lenguaje evita tecnicismos; la geolocalización se pide cuando aporta valor; la corrección de ingredientes ocurre justo antes del descuento.

Feedback incorporado durante el desarrollo

Observación real	Cambio realizado	Aprendizaje
El buscador quedó más profesional visualmente pero dejó de devolver resultados.	Se restauró el comportamiento, se separó banco/tipo y se añadieron alias y pruebas.	Una mejora estética no puede degradar el flujo principal.
Galicia crédito y débito no mostraba supermercados con descuentos.	Se priorizaron fuentes de supermercados y se complementó con bancos.	La ausencia de resultados debe distinguirse de ausencia de beneficios.
Los calendarios comunitarios eran ficticios.	Se migraron a publicaciones reales en D1 con descripción y público.	Los datos sociales necesitan procedencia y privacidad explícitas.
Al cocinar, el inventario no reflejaba el consumo real.	Se agregó confirmación editable de ingredientes antes de descontar.	La última palabra sobre cantidades debe ser del usuario.

Próxima validación recomendada

Realizar cinco pruebas moderadas con personas que vivan solas: crear un plan, corregir un ingrediente, encontrar una promoción, publicar un calendario y borrar memoria. Medir éxito de tarea, tiempo, errores y confianza en las recomendaciones.

11. Ciberseguridad y privacidad

Riesgo	Control / decisión
Secretos OAuth/correo expuestos	.env ignorado y variables privadas de producción; rotación si se divulgan.
Acceso horizontal a otro usuario	La identidad se obtiene de la sesión, nunca de un userId enviado por el cliente.
Contraseñas comprometidas	Better Auth administra hashes; la app no almacena contraseñas en su estado.
Datos privados publicados	Comunidad excluye correo, inventario, alergias, ubicación y pagos.
Ubicación sin consentimiento	Permiso explícito, consulta puntual y sin persistencia.
Promoción falsa o vencida	Fuente/vigencia visibles; solo datos compatibles calculan ahorro.
Crawler abusivo / SSRF	HTTPS, dominios oficiales, misma raíz, profundidad, páginas y timeout limitados.
Recomendación alimentaria insegura	Validación antes/después y otra vez al publicar/adaptar.
Stock negativo	Unidades y máximos validados antes de descontar.
Inyección de prompt	No se ejecutan prompts ni LLM; entradas normalizadas como datos.

Prioridad

La seguridad alimentaria domina el ranking funcional. La aplicación nunca debe compensar una alergia o un vencimiento con ahorro, popularidad, gusto o rapidez.

12. Co-work con inteligencia artificial

Herramienta	Uso	Evaluación crítica
ChatGPT Work	Ideación y primeras iteraciones	Aceleró exploración; las ideas se verificaron en código.
Codex en VS Code	Implementación, pruebas, auth, D1, documentación y despliegue	Eficaz para recorrer el repo; requirió supervisión en OAuth y fuentes externas.
Gemini	Consultas puntuales y contraste	Segunda perspectiva; no integra el runtime.

Reflexión

Sin co-work con IA, integrar frontend, nueve agentes, persistencia, autenticación, mapas, promociones y pruebas habría requerido aproximadamente el doble de tiempo. La mayor ganancia fue mantener contexto entre módulos y automatizar verificaciones repetitivas.

La IA también falló: hubo una regresión del buscador bancario, supuestos demasiado optimistas sobre promociones públicas y documentación desactualizada. El proyecto mejoró cuando esos resultados se trataron como hipótesis y se contrastaron con pruebas, código, fuentes oficiales y feedback humano.

Criterio profesional

La IA fue copiloto de desarrollo, no fuente de verdad del dominio. Las decisiones sobre alergias, privacidad, autenticación, vigencia de promociones y publicación se fijaron mediante reglas explícitas y controles verificables.

13. IA local: papel y aporte al usuario

1. Papel de un LLM/SLM local

Un modelo pequeño local podría actuar como componente de soporte, no como autoridad sobre alergias o vencimientos. Interpretaría descripciones libres de inventario, resumiría feedback semanal y explicaría recetas con lenguaje flexible. Los nueve agentes determinísticos conservarían las decisiones críticas por ser auditables y reproducibles. No reemplazaría una API externa actual —MealBoard no consume un LLM en producción— sino que agregaría comprensión de lenguaje sin costo por token.

2. Aporte al usuario

El usuario podría escribir “me queda medio paquete de arroz y algo de pollo” o pedir una explicación adaptada a su experiencia. El procesamiento local mejoraría privacidad y funcionamiento sin internet para tareas de texto; mapa y promociones seguirían necesitando conexión. Toda propuesta pasaría después por las reglas de alergias, vencimientos y stock.

Arquitectura hipotética

Texto libre → SLM local interpreta → Reglas MealBoard validan → Usuario confirma

El modelo local nunca escribe directamente en D1 ni decide una sustitución segura. Solo produce una propuesta estructurada que el motor de reglas valida.

14. IA local: aporte profesional y límites

3. Aporte profesional

Permitiría analizar localmente logs anonimizados, dificultades frecuentes y patrones de abandono sin enviar datos sensibles fuera de la organización. Facilitaría clasificar feedback, preparar reportes y probar nuevos flujos sin conexión. También exigiría evaluaciones propias para medir calidad, sesgos y errores en el dominio alimentario.

4. Limitaciones frente a la nube

La calidad depende de memoria, CPU o GPU. Un modelo pequeño comprende menos matices y puede inventar información; no debería calcular alergias, nutrición ni promociones sin validación posterior. Además hay que descargar, versionar, actualizar y monitorear el modelo. Una API cloud suele ofrecer mayor capacidad y mantenimiento centralizado, a cambio de costo, conexión y exposición de datos.

Experimento opcional

Ejecutar Ollama con un SLM y pedir una adaptación lingüística de receta, sin delegar decisiones de alergias. Capturar modelo, prompt, respuesta, hardware y tiempo. El experimento queda deliberadamente fuera del runtime publicado.

Conclusión

MealBoard ya cumple su objetivo mediante una arquitectura explicable, autenticada y persistente. El siguiente salto de calidad no depende de agregar generación libre, sino de validar con usuarios, ampliar datos oficiales estructurados y observar métricas de éxito sin comprometer privacidad.

Checklist para defensa oral

- Abrir repositorio y producción desde la portada.
- Explicar el problema y la prioridad de alergias/vencimientos.
- Mostrar el flujo de nueve agentes y una traza Observó/Decidió/Entregó.
- Completar el ciclo: generar, confirmar, cocinar, descontar y compartir.
- Cerrar con UX, seguridad, límites de datos y posible SLM local.